Real Coder (mattock_name) Reviewer Guidelines

📂Changelog

Date	Description
Mar 15, 2026	Guidelines First Draft

🌟 Overview

🎉 **Congratulations! You've been selected to join the Reviewer Team** on this project! Reviewers play a crucial role in ensuring the highest quality standards. You are the first line of defense, and it is your responsibility to ensure that the tasks you review meet the project's expectations!

The internal team will actively monitor your performance, providing feedback to help you grow and succeed in this role.

Make sure to always read the **attempter's instructions** carefully.

Next steps: Read the evaluation tables and follow the instructions. These guidelines are essential for maintaining consistency, quality, and fairness across the review process.

📖 Table of Contents

📂Changelog
🌟 Overview
📖 Table of Contents
📚 Resources
🛠️ Core Skills for Success
🧨 Summarized Task Workflow

Outlier

latam.coder1946@remotasks.com+outlier | 2026-04-20T17:55:21.216Z CONFIDENTIAL

🖌️ Review Workflow
1. Create Environment 🧑‍💻
📂 without_solution
📂 with_solution
📁 Expected Directory Structure
2. Run Eval ⚙️
Tara Eval: Section Breakdown
"Task Info" Section
"Eval" Section
3. Fix Task 🛠️
3.1 Review Re-written Prompt
Handling Fix Scope:
3.2 Review Unit Tests
Handling Fix Scope:
3.3 Review Rubrics
Handling Fix Scope:
3.4 Review Golden Patch & Complete Coverage
Coverage check:
Handling Fix Scope:
4. Run Validation.sh Again!!
Final Environment Checklist
Verification Goals
5. Score and Feedback
Helpful Feedback
📏 Reviewer Grading Rubric
Mandatory Grading Reference
Key Grading Principles
How to Navigate the QC Doc

📚 Resources

- Project Instructions
- Tara Eval App

Off-Platform Linter/System Prompt to check Golden Patch, Unit Tests, Rubrics:

- Tutorial on How to Use it
- System Prompt Folder
- Another System Prompt Folder
- Coverage System Prompt

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-20T17:55:21.216Z

🛠️ Core Skills for Success

To excel in this role, you should have:

- **Attention to detail:** Detect subtle errors in code, logic, tests, and rubrics.
- **Strong written communication:** Provide clear and structured corrections.
- **Objective judgment:** Evaluate work based on project requirements.
- **Knowledge of the scoring rubric:** Ensure proper application of grading criteria.

🖍️ Summarized Task Workflow

1. **Set up Environment**: On your local machine, create a dedicated folder with the name of your choice for the review task, Download all the necessary files like tests.zip, codebase.zip, run.sh, parsing.sh, and requirements.txt/package.json (if applicable).
2. **Run Tara Eval**: Use the *Tara Eval* tool. It will provide an initial quality report and flag guideline violations.
3. **Fix Task**: Manually review the entire task. You are responsible for fixing any issues you find.
4. **Run Validation.sh again**: As a final check, re-run the validation.sh to make sure your task passes the f2p tests completely.
5. Submit the task! 🎉

🖌️ Review Workflow

After an attempter completes a task, a reviewer (like you) will evaluate it based on accuracy, adherence to guidelines, and task quality. If something is wrong in the task, you need to fix it!

1. Create Environment 🧑‍💻

On your local machine, create a folder with a name of your choice (e.g., `review_task_1`, `task_2`, etc.).

Inside this folder, create a main directory named **`app`**. Download all the required files for the task (`run.sh`, `parsing.sh`, `Dockerfile`, `codebase.zip`, and `tests.zip`) and place them inside the `/app` directory.

Once done, you can create two additional folders:

- **`without_solution`**
- **`with_solution`**

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-20T17:55:21.216Z

📂 without_solution

Add the following:

- `Dockerfile`
- `run.sh`
- `parsing.sh`
- `tests/` folder (after extracting `tests.zip`)

> This setup is used for **baseline execution**, ensuring that all tests fail before implementing the **Golden Patch**.

📂 with_solution

Add the following:

- All extracted files from `codebase.zip`
  - `Dockerfile`
  - `run.sh`
  - `parsing.sh`
  - `tests/` folder
  - Golden Patch solution files

> This setup is used for **verification execution**, ensuring that the attempter's **Golden Patch** passes all test cases.

📁 Expected Directory Structure

```python
review_task/
│
├── app/
│   ├── run.sh
│   ├── parsing.sh
│   ├── Dockerfile
│   ├── codebase.zip
│   └── tests.zip
│
├── without_solution/
│   ├── run.sh
│   ├── parsing.sh
```


CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-20T17:55:21.216Z

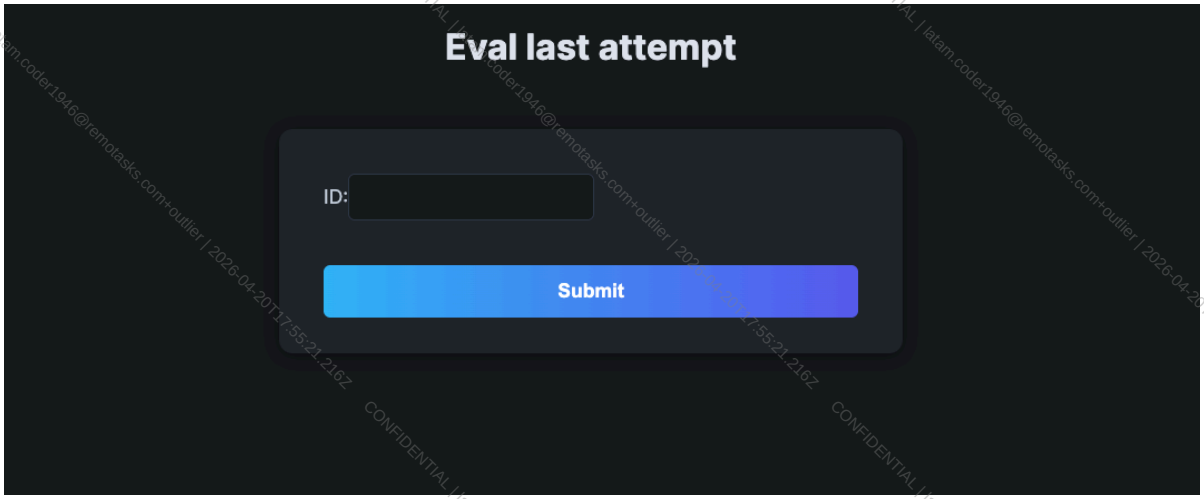
```
│    ├── Dockerfile
│    └── tests/
│
└── with_solution/
     ├── run.sh
     ├── parsing.sh
     ├── Dockerfile
     ├── tests/
     └── (extracted codebase files)
```

You can now open your review task folder in your preferred IDE (Cursor, VS Code, Opencode, etc.).

2. Run Eval ⚙️

A **mandatory** step in the review process is to use the Tara Eval tool, which runs quality checks on the task you are reviewing.

1. Navigate to the **Tara Eval** app: https://taraeval.vercel.app/
2. Enter the **Task ID** of your current task.
3. Click **"submit"** to run it and wait for the checks to complete.
4. **Verify the results.** Confirm that the task passes all quality checks. If any checks fail, you need to address them. Below is an explanation of the eval results.



> **Warning**
> The tool is a **guide**! Do **NOT** use it as a source of truth.

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-20T17:55:21.216Z

> Use the tool to identify potential issues and automate common checks.
> You are responsible not only for reviewing the task but also for **making any necessary fixes** to ensure it meets all quality guidelines.

Tara Eval: Section Breakdown

The Tara Eval tool is designed to help you quickly assess the quality of a submission. It's divided into several key sections, each providing specific insights into the task.

"Task Info" Section

This section provides a high-level overview of the task and the identifiers associated with the current attempt.

- **Metadata:** Displays key identifiers for the task.
  - **Task ID:** The unique identifier assigned to the task.
  - **Attempt ID:** The unique identifier assigned to the specific submission attempt.
- **Attempt**: This section contains all materials submitted for evaluation.
  - **Prompt:** Displays the exact re-written prompt submitted by the attempter which was used to generate the Golden Patch.
  - **Rubrics:** Lists the evaluation criteria used to assess whether the Golden Patch follows the re-written prompt instructions. Each rubric includes a description of the requirement, its assigned weight, and the evaluation dimension.
  - **Unit Test:** Displays the results of automated tests executed on the submission. Each test shows the test name and whether the result **PASSED** or **FAILED**.

"Eval" Section

This is the main analysis section of the tool. It provides a comprehensive breakdown of the task's execution, structure, and quality. It contains the following components:

Eval Execution [PASS/FAIL]:

This step runs the `validation.sh` script inside a sandbox environment to confirm that the task behaves correctly before and after applying the Golden Patch.

- **Execution Output:** Displays the logs produced when `validation.sh` is executed in the sandbox environment.
- **Eval attempt before.json == eval before.json [PASS/FAIL]:** Compares the `before.json` generated during the Eval run with the expected `before.json`. This verifies that the base repository fails the unit tests as expected.

- **Eval attempt after.json == eval after.json [PASS/FAIL]:** Compares the `after.json` generated during the Eval run with the expected `after.json`. This verifies that the Golden Patch fixes the implementation and all unit tests pass.

Eval Golden Patch [1/2/3/4/5]:

This section evaluates how the submitted Golden Patch satisfies the prompt requirements after evaluation.

- **Overall Score [1/2/3/4/5]:** Displays the final evaluation score assigned to the Golden Patch based on requirement satisfaction.
- **Total Requirements:** Indicates the total number of requirements extracted from the task prompt and rubric definitions.
- **Requirement Satisfaction Summary:** Indicates how the evaluated requirements are distributed across **fully satisfied**, **partially satisfied**, and **not satisfied** categories based on how well the implementation meets the expected behavior.
- **Overall Explanation:** Provides a brief summary explaining how the Golden Patch performs with respect to the task requirements and expected functionality.
- **Requirements:** Lists the individual requirements evaluated for the Golden Patch and their respective evaluation outcomes.

> **Note:** It's observed that **Tara** only flags and creates requirements from the **Expected Interface** of the prompt and **not description, key requirements, deliverables,** so make sure to keep a note of those asks too.

Eval Verification

This section assesses whether the task has sufficient automated checks to reliably validate the requirements and ensure the correctness of the solution.

- **Eval Tests:** Evaluates the unit tests associated with the task. This section summarizes the **total number of tests**, along with how many **passed**, **failed**, or are **missing**. Failed tests indicate issues with the implementation or expected behavior, while missing tests highlight prompt requirements that may not be fully verified through unit tests.
- **Eval Rubrics:** Reviews the rubric-based evaluation associated with the task requirements. This section reports the **total number of rubrics** and their outcomes, including **passed**, **failed**, or **missing** rubrics. It helps determine whether the defined evaluation criteria are properly validated and whether each requirement is adequately assessed.

- **Eval Coverage [🟢/🟡/🔴]:** Measures how well the task requirements are supported by **tests** and **rubrics** using a dual-layer coverage approach. Each requirement must be fully validated by **at least one layer** (tests or rubrics), and does not need to be covered by both. Coverage is categorized as **Good (>75%)**, **Mid (>=50%)**, or **Low (<50%)** based on how many requirements have sufficient validation. This section also includes a requirement-level breakdown showing **coverage status**, along with the specific rubrics and/or tests used to verify each requirement. Measures how well the task requirements are supported by both **tests** and **rubrics**.

3. Fix Task 🛠️

Once you've setup the environment, and read through the Tara Eval results, you can now manually review the task.

3.1 Review Re-written Prompt

Carefully review the re-written prompt to ensure it fully captures the intended task description and aligns with the evaluation criteria.

- **Instruction Completeness:**
  Verify that all required elements from the original task are present, including the **expected interface**, inputs/outputs, and overall functionality.

- **Clarity & Specificity:**
  Ensure the prompt is clear, unambiguous, and does not leave room for multiple interpretations of the expected behavior.

- **Expected Behavior & Edge Cases:**
  Confirm that all key behaviors are explicitly defined, including any important edge cases or constraints (e.g., filtering logic, status handling, ordering, etc.).

- **Constraints & Feasibility:**
  Check that the constraints are realistic and enforceable (e.g., required libraries, no external dependencies, specific frameworks). Ensure the task is solvable within scope.

- **Alignment with Workflow Criteria:**
  Validate that the prompt satisfies:
  - **No contrived or unnatural requirements**
  - **Truthfulness and technical correctness**
  - **Clear reasoning requirements where applicable**

- **Consistency with Tests & Rubrics:**
  Ensure that everything mentioned in the prompt is testable and reflected in the unit tests and rubrics. There should be no missing or extra requirements.

- **Avoid Over/Under Specification:**
  The prompt should not be overly restrictive (e.g., forcing exact file names unnecessarily) or too vague (missing key implementation details).

Make sure the prompt defines a **complete, testable, and realistic task** before proceeding to the next step.

Important Tips: 💡

- **Use the Prompt Linter:**
  Check the linter available after the prompt to identify inconsistencies, missing elements, or weak instructions.
- **Use off-platform linter on your coding agent IDE (Cursor, opencode etc.):**
  - Create a file named prompt.md in the task folder **and add** the attempter's **rewritten** prompt. Add this file to Cursor's agent window.
  - Use the system prompt named promptchecker.md (available in [System Prompt Folder]) and load it into Cursor.
  - Now add the task description for the task to Cursor and ask it to analyze and highlight missing requirements, inconsistencies, or unclear instructions.
  - Fix accordingly.

Note: 🚨
These tools should be used only as a **guide**, not as the source of truth. You are responsible for ensuring the prompt is correct, complete, and aligned with the task requirements.

Handling Fix Scope:

- If there are major issues in the prompt (**missing requirements**, **contradictions**, **incorrect behavior**, **unclear interface** or **impossible asks** [these are the most common errors observed]), you will need to first fix the entire prompt and then you will need to update all downstream components- including unit tests, Golden Patch, and rubrics to ensure full alignment.

- If there are minor fixes (wording, small clarifications, edge case additions), ensure these changes are consistently reflected downstream, especially in unit tests and rubrics, to avoid mismatches during evaluation.

3.2 Review Unit Tests

Review the unit tests to ensure they correctly enforce the behavior defined in the prompt and follow proper F2P validation.

- **Baseline Execution (F2P Setup – without_solution):**
  Set up the `without_solution` environment and run the tests. All tests should **fail** in this setup, confirming that the tests correctly capture missing

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-20T17:55:21.216Z

functionality.

- **Verification Execution (with_solution):**
  Set up the `with_solution` environment (including the extracted Golden Patch) and run the tests. All tests should **pass**, confirming that the implementation satisfies the expected behavior.

- **Alignment with Prompt:**
  Verify that all key requirements from the prompt are covered by tests. Every important behavior should be validated through at least one test.

- **F2P (Fail-to-Pass) Validation:**
  Ensure tests fail in `without_solution` and pass in `with_solution`. This guarantees determinism and correctness of evaluation.

- **Coverage Completeness:**
  Check that tests cover all core functionality, prompt requirements and important constraints. Identify any missing scenarios and fix them.

- **Correctness of Assertions:**
  Ensure test expectations validate intended behavior and are not incorrect or misleading.

- **Avoid Over-Specific or Fragile Tests:**
  Do not rely on hardcoded values or implementation-specific details unless explicitly required.

- **No Redundant or Irrelevant Tests:**
  Remove duplicate or unnecessary tests that do not add meaningful validation.

Important Tips: 💡

- **Use the Unit Tests Linter:**
  Check the linter available after the unit tests to identify inconsistencies, missing elements, or weak instructions.
- **Use off-platform linter on your coding agent IDE (Cursor, opencode etc.):**
  - Load the tests folder into cursor IDE.
  - Create a file named `overly_specific_check.md` and add the system prompt from [Step 2b (Attempter Guidelines)](#).
  - Run the check to identify overly specific tests, missing scenarios, weak assertions, or incorrect validations.
  - Fix issues accordingly.

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-20T17:55:21.216Z

> **Note:** 🚨
> These linters should be used only as a **guide**, not as the source of truth. You are responsible for ensuring the prompt is correct, complete, and aligned with the task requirements.

Handling Fix Scope:

- If baseline (`without_solution`) does not result in all tests failing, the tests are incorrect and must be fixed.
- If `with_solution` does not result in all tests passing, the issue may lie in the tests or Golden Patch and must be investigated.
- If you fix the prompt, remember to align tests accordingly.
- Strengthen weak tests and relax overly strict or incorrect ones.
- Re-run validation after any changes to ensure proper F2P behavior.

3.3 Review Rubrics

Review the rubrics to ensure they provide a rigorous, implementation-agnostic evaluation of the prompt's requirements that are not covered by unit tests.

- **Coverage & Criticality:** Ensure the rubrics cover the **top 30 most important requirements** (explicit or critical implicit) that cannot be verified by unit tests. There should be **100% coverage** of all prompt requirements when unit tests and rubrics are combined.
- **Atomicity:** Each criterion must check **only one idea** or requirement. Avoid "bundling" unrelated constraints into a single rubric item.
- **Self-Containment:** Criteria must be evaluatable using **only the model response and the rubric text**. A reviewer should not need to refer back to the prompt or external facts to determine if the criterion is met.
- **Implementation Neutrality (Overfitting):** Rubrics must be flexible enough to accept **different valid implementations**. They should not be "overfit" to the Golden Patch (e.g., mandating a specific internal file name or helper function not required by the prompt).
- **Framing & Accuracy:** All criteria must be **positively phrased** (evaluating to "True" or "Yes" for a good response). Ensure there are no **objectively wrong**, irrelevant, or counterproductive criteria.
- **Redundancy:** Identify and remove **overlapping or redundant** criteria where multiple items check the same requirement or one is a subset of another.
- **Categorization & Weighting:** Verify that each rubric is assigned to one of the **five dimensions**: Instruction Following, Code Correctness, Quality, Clarity, or Efficiency. Weights must strictly be **1 (Nice to have), 3 (Important), or 5 (Mandatory)**.

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-20T17:55:21.216Z

Important Tips: 💡

- **Use the Rubrics Linter:**
  Check the linter available after the rubrics to identify inconsistencies, missing elements, or weak instructions.
- **Use off-platform linter on your coding agent IDE (Cursor, opencode etc.):**
  - Create a file named rubrics.md and add all your rubrics along with their dimension and weights.
  - Load both tests folder and rubrics.md file into Cursor.
  - Create a file named `rubrics_and_test_coverage.md` and add the system prompt from [Step 3b (Attempter Guidelines)](#).
  - Run the check to identify coverage gaps, atomicity issues, or overly specific criteria and tests.
  - Ensure the combined suite achieves 100% coverage of all prompt requirements.
  - Fix any identified issues, such as **overlapping criteria** or **poorly framed** (subjective) rubrics, accordingly.

Note: 🚨
These linters should be used only as a **guide**, not as the source of truth. You are responsible for ensuring the prompt is correct, complete, and aligned with the task requirements.

Handling Fix Scope:

- If you fix the prompt, update the rubrics to maintain **100% coverage** of all requirements.
- Relax overly specific or overfitted criteria to ensure they are implementation-agnostic and won't fail valid solutions.
- Split non-atomic "bundled" requirements and ensure all criteria are self-contained and evaluatable from the response alone.
- Correct miscategorized criteria and ensure all weights strictly follow the **1, 3, or 5** scale.
- Remove redundant or overlapping rubrics, particularly those already sufficiently verified by unit tests.

3.4 Review Golden Patch & Complete Coverage

Review the Golden Patch to ensure it provides a high-quality "ground truth" implementation and verify that the combined evaluation suite achieves 100% prompt coverage.

Golden Patch Integrity:

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-20T17:55:21.216Z

- **Functional and Technical Accuracy** requires the solution to answer every requirement in the rewritten prompt while executing perfectly within the Ubuntu 22.04 Docker container without runtime errors.
- **High-Quality Code Standards** must be maintained by following fundamental principles like modularity and abstraction while avoiding hard-coded values or fragile input assumptions.
- **Optimization and Asset Compliance** ensure that algorithms are well-optimized and all visual assets are 100% commercially free, strictly avoiding any content from Unsplash.

Holistic Coverage Verification:

- **100% Requirement Mapping** ensures every explicit and critical implicit requirement in the prompt is verified by either a unit test or an atomic rubric criterion.
- **Backend and Frontend Split** dictates that backend behavior be primarily covered by unit tests, while UI/UX and qualitative traits are verified through rubrics.
- **Interface and Agnostic Testing** requires that the "Expected Interface" be fully tested through a "black-box" approach that does not penalize valid alternative implementation paths.

Coverage check:

To ensure your task meets the **100% coverage** mandate, you must perform a programmatic audit of your **Golden Patch** against your unit tests and rubrics. This step identifies any "orphan" requirements that are neither tested by code nor evaluated by a rubric.

Why use this? This check prevents manual oversight by cross-referencing every functional, technical, and constraint-based requirement in your prompt with your verification suite. It ensures that the combined evaluation (tests + rubrics) leaves no gaps in verifying the model's response.

How to Use the Coverage Checker:

1. Download the **Coverage Check System Prompt** (SP) from **here**. Create a file named coverage_audit.md and add the SP in the file. Once done, load it in your Cursor or OpenCode environment.
2. Now, add the .md file, rewritten prompt, tests folder, codebase (Golden Patch), and rubrics. Then ask the model to provide the complete audit on the same.
3. **Next, analyze the** generated **Requirements Coverage Matrix**. If the status is **FAIL**, identify the specific requirements marked as **MISSING**.
4. For every missing requirement, either add a **unit test** (for backend behavior) or a **rubric criterion** (for frontend/qualitative traits) and fix the failing issues.

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-20T17:55:21.216Z

> **Note:** Reiterate this process multiple times to ensure 100% coverage of Golden Patch through unit tests and rubrics.
>
> **Disclaimer:** This linter acts as a guide only; do not rely on it as the sole source of truth. You remain responsible for the final accuracy and completeness of the task.

Handling Fix Scope:

- If the Golden Patch fails any criteria, debug the implementation until it passes 100% of the unit tests and rubrics.
- Update the **Golden Patch** and re-verify the task if you discover untestable requirements or need additional **Docker** dependencies.
- Confirm the F2P tests show "**FAILED**" before **Golden Patch** implementation and "**PASSED**" status for every test case after applying the patch.
- Re-run the coverage linter to ensure no requirements were dropped following any implementation or verification changes.
- Verify that the final codebase ZIP is correctly structured without nested folders to ensure reproducibility.

4. Run Validation.sh Again!!

Before final submission, you must re-run the validation.sh script to ensure all manual fixes are correctly integrated and reproducible in a clean environment. This step is critical for verifying that your task is deterministic and follows strict project data specifications.

Final Environment Checklist

- **Strict Directory Structure:** Ensure the /app directory contains only the mandatory files: Dockerfile, tests.zip, codebase.zip, run.sh, and parsing.py.
- **Script Integrity:** Double-check that run.sh, parsing.py, and the Dockerfile are configured according to provided templates. Do not modify any "DO NOT MODIFY" sections.
- **Implementation Alignment:** Verify that the logic in run.sh (test execution commands) and parsing.py (test result parsing) is perfectly aligned with your codebase and test suite. All scripts must run consistently and properly against your specific implementation.
- **ZIP Content:** Verify that codebase.zip contains files directly at the root (no parent folder) and tests.zip contains the tests/ folder first.
- **Updated Artifacts:** Ensure the validation is run against your **most recently fixed** versions of the prompt, unit tests, Golden Patch, and rubrics.

Verification Goals

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-20T17:55:21.216Z

- **Deterministic Results:** Your before.json must show 100% of tests as **FAILED** (not errored), and your after.json must show 100% as **PASSED**.
- **Timestamp Accuracy:** After a successful run, confirm the "last edited" timestamps for all artifacts match your final session to avoid submitting outdated files.

5. Score and Feedback

In this step, you will assess the quality of the task to ensure it aligns with the project guidelines. Your goal is to provide constructive feedback and apply the scoring rubric consistently (Grading rubric).

Helpful Feedback

Providing **clear, detailed feedback** is crucial for maintaining **high-quality** reviews and tasks. Feedback should help Attempters improve by explaining what went wrong and **how to fix it**. Keep in mind that some Attempters may be new to the task or struggling with certain concepts—your feedback should **guide them toward improvement** rather than simply pointing out mistakes.

📏 Reviewer Grading Rubric

This final section defines the standards for determining the ultimate score of the task. Reviewers must adhere strictly to the specialized criteria and calculation methodologies established for this project.

Mandatory Grading Reference

All grading decisions **must** be based solely on the official documentation. Access the comprehensive grading framework here:

Official Real Coder QC Specification & Grading Rubrics

Key Grading Principles

- **Strict Adherence:** Reviewers are required to be exceptionally rigorous. Do not use outside intuition or "gut feelings"; grade exclusively based on the metrics, error categories, and scoring rules defined in the linked QC Spec.
- **The "Lowest Dimension" Rule:** The final task grade is determined by the lowest score received across any individual dimension. For example, if Instruction Following is a 2, the entire task must be rated a 2, regardless of other high-performing areas.

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-20T17:55:21.216Z

- **Holistic Error Tallying:** You must use the specific percentage-based thresholds (e.g., the 5% Major Error rule) for rubric quality to determine if a task passes or fails.

How to Navigate the QC Doc

To grade efficiently, focus your review on these core sections of the document:

- **Audit Workflow (Steps 1–6):** Follow the step-by-step checklist to evaluate the Prompt, Golden Patch, F2P Tests, and Rubrics in the correct order.
- **Dimension Rubrics:** Refer to the detailed tables for **Prompt Quality**, **Expected Interfaces**, and **Response Readability/Design** to identify specific "Fail" vs. "Non-Fail" triggers.
- **Appendix – Rubric Quality Definitions:** Use this section to classify contributor errors as **Major**, **Moderate**, or **Minor** before calculating the final holistic score.

> **Reminder:** Consistency is paramount. Re-read the QC Spec before every audit to ensure you are applying the most current standards for atomicity, self-containment, and implementation neutrality.

CONFIDENTIAL | latam.coder1946@remotasks.com+outlier | 2026-04-20T17:55:21.216Z